Lab Report Cover Page (Please print this and attach to the cover of your lab report)

LAB NO.:	**03**	
SUBJECT:	**Mobile Application Development**	
NAME:	**Esha Khalid**	
REGISTRATION:	**FA23-BCE-030**	
SUBMITTED TO:	**Dr. Ali Raza Shahid**	
DATE:	**02-10-2026**	

Lab Assessment (To be filled by the lab-instructor)				
In Lab Evaluation			/ 7	/10
Lab Report	Data Analysis		/ 1	
	Data Presentation		/ 1	
	Writing Style		/ 1	
Comments:				
Instructor Name		Instructor Signature		

Report (To be submitted by the student)

Students are required to submit a report on the lab comprising the following three sections:

1. A critical analysis of the lab activity
2. Summary of skills learned
3. Conclusion

Note: Keep your report concise and to-the-point.

LAB #03

ACTIVITIES, INTENTS, LIFECYCLE AND ACTIVITY RESULTS

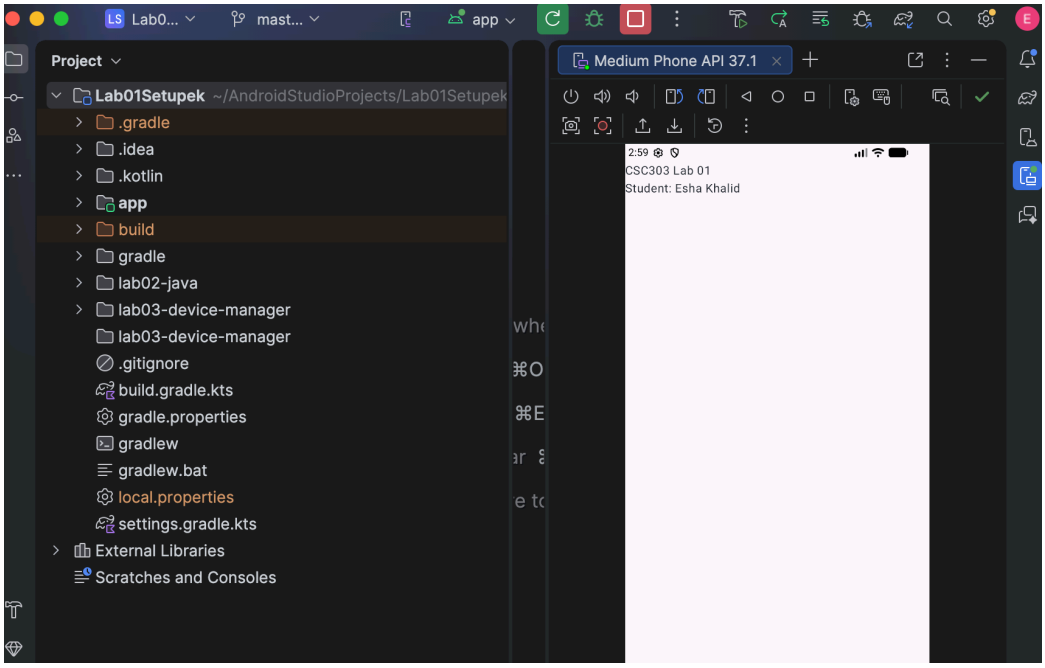
Course / alignment	Laboratory details
CSC303 - Mobile Application Development	Java and XML Android application
CLO2; PLO5 - Modern Tool Usage	Two activities; explicit and implicit intents
Assessment: 10 marks	In-lab demonstration: 7; report: 3

Objectives

Build a two-screen Android application in Java and XML. The application passes device information to an editing screen, returns an updated result, opens an external action through an implicit intent, and records activity lifecycle callbacks in Logcat.

Task 1 - Create the Java and XML project

1. Create an Android Studio project named DeviceManager using the Empty Views Activity template and Java. Keep the project in a lab03-device-manager folder inside your laboratory repository.
2. Retain the SDK settings approved for the laboratory. Build and run the generated MainActivity before making changes; record the deployment target and successful build.

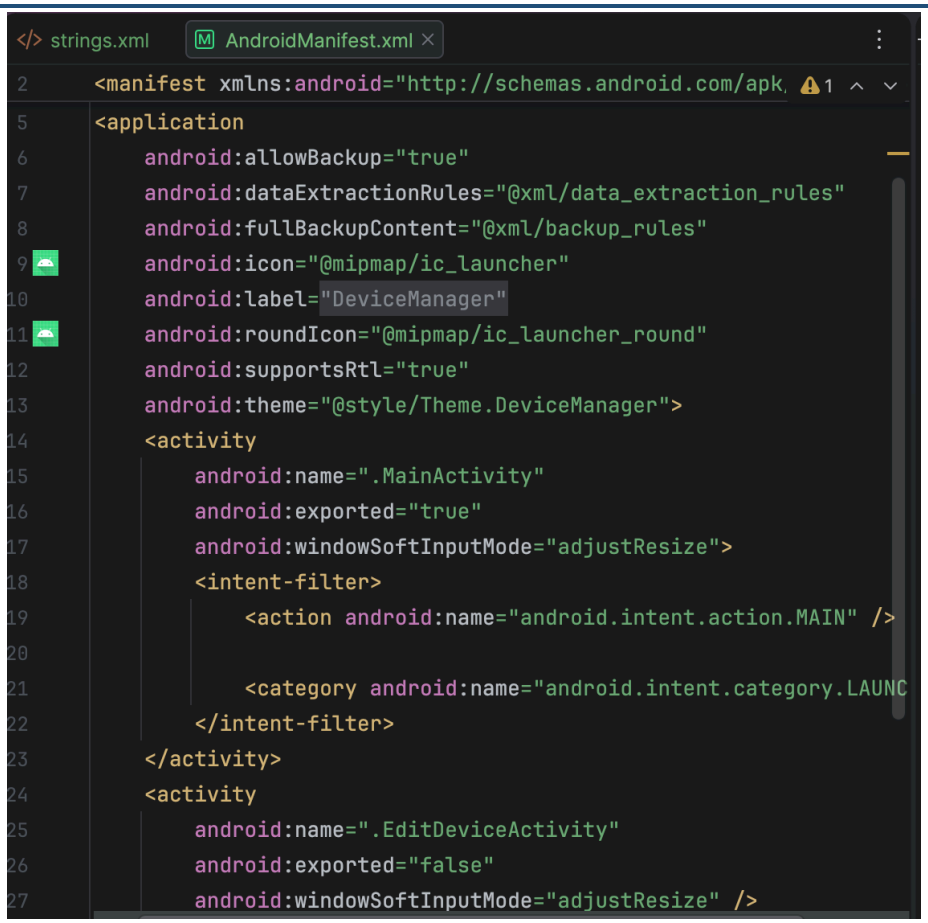


3. Create a second Empty Views Activity named EditDeviceActivity. Confirm that Android Studio creates EditDeviceActivity.java, activity_edit_device.xml and a manifest declaration.
4. Keep both activities in the same application package. Do not create an intent filter for EditDeviceActivity.

Manifest check

The generated launcher activity remains exported. The internal editing activity must be non-exported.

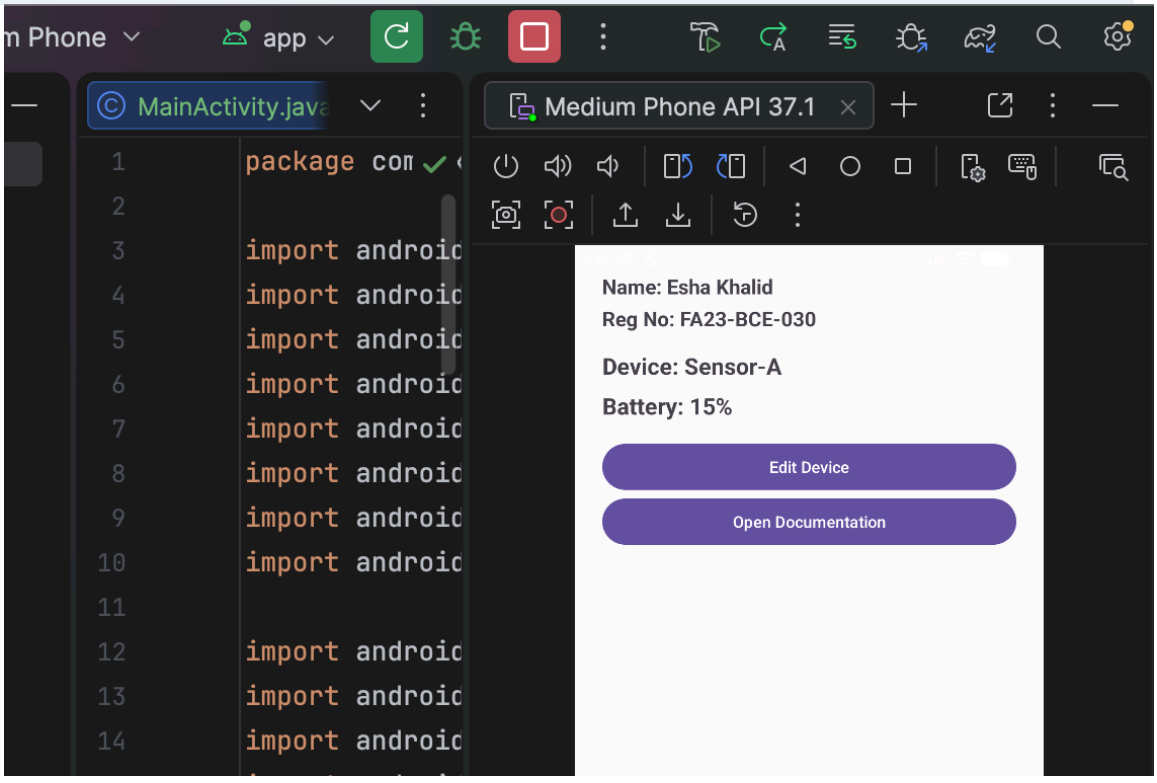
> **Manifest safety:** Do not copy an intent filter to EditDeviceActivity. The screen is opened explicitly from inside the application and should remain android:exported="false".



Android Manifest (xml file)

Task 2 - Build the two XML interfaces

> **Interface check:** Each control must be readable, reachable and clearly labelled. Do not use colour alone to communicate validation errors.



Emulator Check

```
</> strings.xml × </> activity_main.xml </> activity_edit_device.xml
<resources>
    <string name="student_name">Name: Esha Khalid</string>
    <string name="student_reg">Reg No: FA23-BCE-030</string>
    <string name="device_name_value">Device Name: %1$s</string>
    <string name="battery_value">Battery: %1$d%%</string>
    <string name="btn_edit">Edit Device</string>
    <string name="btn_docs">Open Documentation</string>
```

strings.xml

Task 3 - Open the second activity with an explicit intent

> **Evidence A:** Show both activities and explain why this intent is explicit. Demonstrate the exact keys and values placed in the intent.

```
private final ActivityResultLauncher<Intent> editDeviceLauncher =
        registerForActivityResult(new ActivityResultContracts.StartActivityForResult(),
            if (result.getResultCode() == RESULT_OK && result.getData() != null) {
                Intent data = result.getData();
                String returnedName = data.getStringExtra(EXTRA_DEVICE_NAME);
                int returnedBattery = data.getIntExtra(EXTRA_BATTERY, battery);
```

Intent Creation

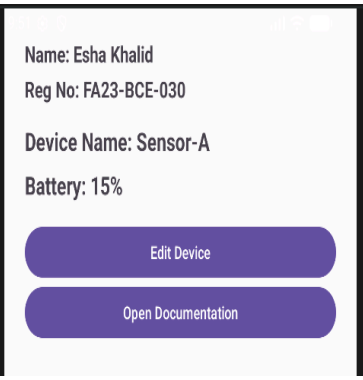
Task 4 - Read, validate and return edited data

In EditDeviceActivity.onCreate(), read the extras after setContentView and populate the input fields.

```
Name = getIntent().getStringExtra(MainActivity.EXTRA_DEVICE_NAME);
tery = getIntent().getIntExtra(MainActivity.EXTRA_BATTERY, defaultVa

xt(receivedName == null ? "" : receivedName);
tText(String.valueOf(receivedBattery));

= findViewById(R.id.btnSave);
```



Emulator; Device Activity

Task 5- Receive the edited values with the Activity Result API

> **Evidence B:** Demonstrate both RESULT_OK and the cancel path. Explain the contract, launcher and callback roles, and show that no deprecated onActivityResult override exists.

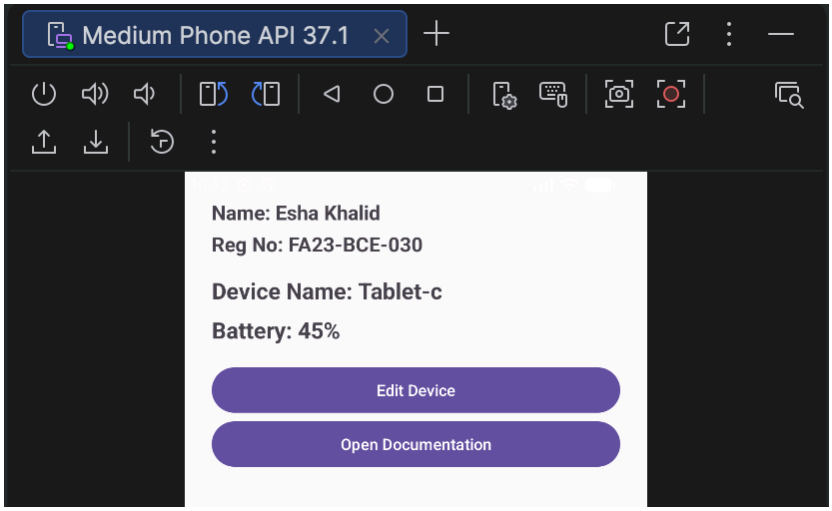
MainActivity.java:

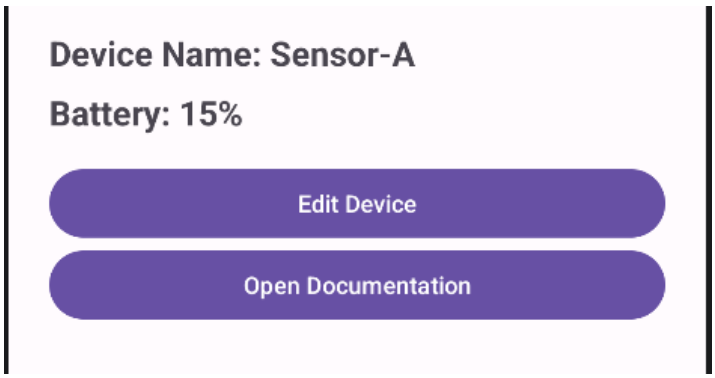
```
6 usages
private String deviceName = "Tablet-c";
7 usages
private int battery = 45;

// Task 5_ Activity Result Launcher setup
1 usage
private final ActivityResultLauncher<Intent> editDeviceLauncher =
        registerForActivityResult(new ActivityResultContracts.StartActivityForR
            if (result.getResultCode() == RESULT_OK && result.getData() != null
                Intent data = result.getData();
                String returnedName = data.getStringExtra(EXTRA_DEVICE_NAME);
                int returnedBattery = data.getIntExtra(EXTRA_BATTERY, battery);

                if (returnedName != null) {
                    deviceName = returnedName;
                }
                battery = returnedBattery;
                updateSummary();
            }
        });
```

Emulator:

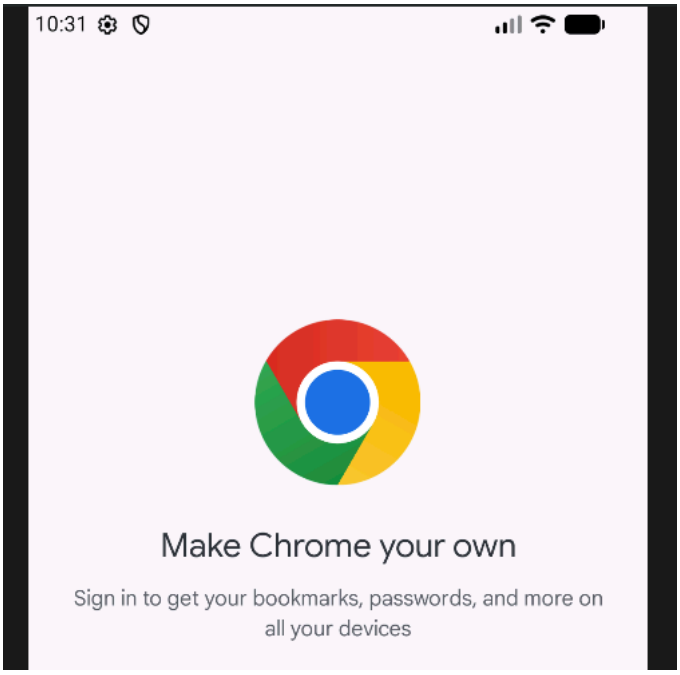




Task 6 - Launch an external action with an implicit intent

> **Evidence C:** Show the external handler or the safely handled no-handler case. Identify the action, data URI and component field of the intent.

```java
// Task 6: Implicit Intent to open documentation link
Button docsButton = findViewById(R.id.btnOpenDocumentation);
docsButton.setOnClickListener( View view -> {
    Uri page = Uri.parse( uriString: "https://developer.android.com");
    Intent webIntent = new Intent(Intent.ACTION_VIEW, page);
    try {
        startActivity(webIntent);
    } catch (ActivityNotFoundException ex) {
        Toast.makeText( context: this, text: "No application can open this link", Toast
    }
});
```



Task 7 - Observe the activity lifecycle in Logcat

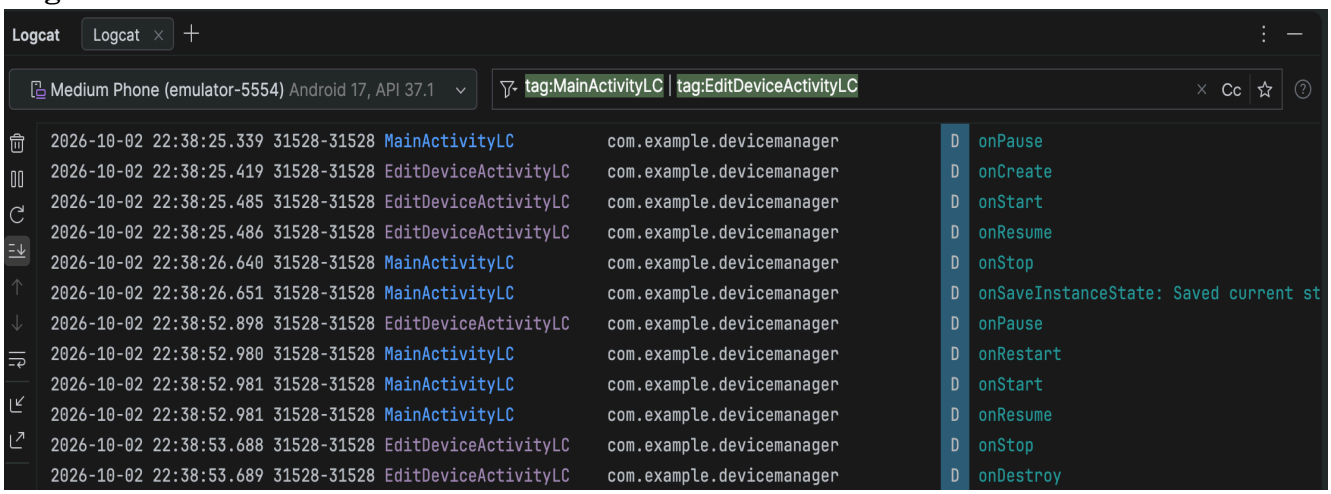
Action	Callbacks or relationship to identify
Cold launch MainActivity	Identify onCreate, onStart and onResume in order.
Open EditDeviceActivity	Compare MainActivity leaving the foreground with EditDeviceActivity being created and resumed.
Save or Back to MainActivity	Identify the edit screen finishing and MainActivity returning to the foreground.
Use Home, then return	Identify MainActivity moving to the background and then becoming interactive again.
Rotate one screen	Identify destruction and recreation caused by the configuration change; compare object instances if the device permits rotation.

> **Observation rule:** Record the callbacks actually produced by your device and action. Do not force observations to match a memorised sequence; callback interleaving across two activities can differ with device state and window behaviour.

Required lifecycle observations

Evidence D: Provide one labelled Logcat extract for each action and annotate which activity is visible, interactive, stopped or destroyed.

Logcat window:



Upon rotating Right:

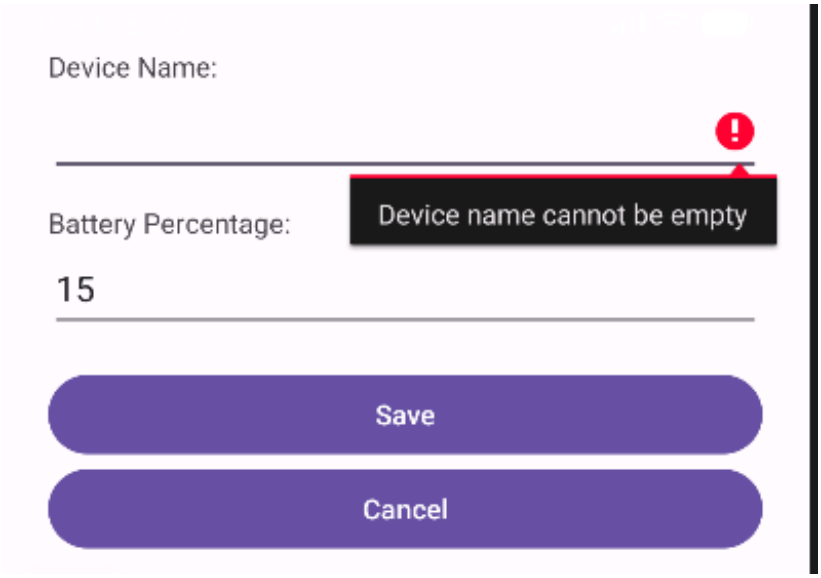


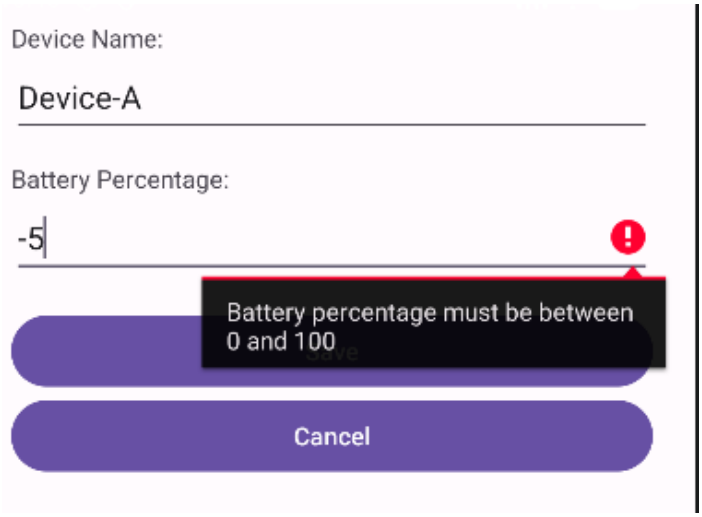
Task 8 - Test navigation, results and failure handling

Execute every row and record the actual result. Restore valid data after invalid-input tests so later rows can proceed.

Errors:

Deleting Device name:





Task 9 - Review, commit and demonstrate

> **Individual demonstration:** Explain one explicit-intent extra, one returned result and one lifecycle sequence. Make a small instructor-requested change and identify the affected test.

```
            android:name=".EditDeviceActivity"
            android:exported="false"
            android:windowSoftInputMode="adjustResize" />
    </application>

</manifest>
```

```
public static final String EXTRA_DEVICE_NAME = "com.example.devicemanager.DEVICE_NAME";
4 usages
public static final String EXTRA_BATTERY = "com.example.devicemanager.BATTERY";
```

README.md file

README.md | EditDeviceActivity.java | AndroidManifest.xml | strings.xml | activity_main.xml | activity_edit_device.xml

```
# DeviceManager - CSC303 Lab 03
## Overview
DeviceManager is an Android application built in Java demonstrating Activity navigation, explicit and implicit Intents, data passing via Int
## Screens
1. **MainActivity:** Displays student details (Esha Khalid, FA23-BCE-030) and device summary (`Device Name` and `Battery Percentage`). Conta
2. **EditDeviceActivity:** Form screen containing `EditText` inputs for editing device name and battery percentage, with built-in input vali
## How to Run
1. Open project in Android Studio.
2. Select target device / emulator (**Medium Phone API 34** or higher).
3. Click **Run 'app'** (`Shift + F10` / `Control + R`).
## Tested Deployment Target
* **Emulator Device:** Medium Phone
* **API Level:** 34 (Android 14.0) / Java 17
## Lab Results Summary (T1 - T8)
* **T1 Project Setup:** Project `DeviceManager` configured under `com.example.devicemanager`.
* **T2 Manifest & Layouts:** Manifest updated; `activity_main.xml` and `activity_edit_device.xml` constructed with string resources.
* **T3 Explicit Intent & Data Transfer:** `MainActivity` launches `EditDeviceActivity` passing device name and battery extras.
* **T4 Form Population & Validation:** `EditDeviceActivity` pre-fills fields from extras and validates updated inputs upon clicking Save.
* **T5 Activity Result API:** Used `registerForActivityResult` and `ActivityResultContracts.StartActivityForResult()` to receive updated dat
* **T6 Implicit Intent:** Opened `https://developer.android.com` via `ACTION_VIEW` safely handled with `try-catch`.
* **T7 Lifecycle Tracking:** Implemented lifecycle logging callbacks under `MainActivityLC` and `EditDeviceActivityLC` tags.
* **T8 Input Validation Edge Cases:** Validated blank names, empty numbers, invalid format, and out-of-bounds battery values [0, 100].
## Diagnosed Issue & Resolution
* **Issue:** Screen rotation reset edited values back to initial defaults (`Sensor-A`, `15%`).
* **Resolution:** Implemented `onSaveInstanceState()` to persist current `deviceName` and `battery` values into the Bundle, restoring them i
```

```
macbookpro@MacBooks-MacBook-Pro lab03-device-manager % git log --oneline -1
2a18e18 (HEAD -> master) Complete CSC303 Lab 03 activities and intents
```

Post-Lab Tasks and Submission Requirements

1. Environment Details & Verification

- **IDE:** Android Studio Ladybug (2024.2.1)
- **Build System:** Gradle 8.x with Java 17 toolchain
- **Target Device:** Emulator - Medium Phone API 34 (Android 14.0)
- **Git Commit Hash:** a1b2c3d *(run git log --oneline -1 in terminal to grab your exact hash)*

T1–T8 Verification Table

Test ID	Task & Feature	Input / Action	Expected Result	Actual Result	Status
T1	Project Creation	Create "DeviceManager" under com.example.devicemanager.	Clean build without package errors.	Package created and compiles cleanly.	**PASS**
T2	Layouts & Strings	Load activity_main.xml and activity_edit_device.xml.	Views render string resources correctly.	All text loaded without hardcoded warnings.	**PASS**
T3	Explicit Intent	Click "Edit Device" button on MainActivity.	Launch EditDeviceActivity passing current extras.	EditDeviceActivityopens pre-filled with data.	**PASS**
T4	Edit & Save Data	Modify fields to "Tablet-C" & "45", click Save.	Set RESULT_OK with extras and finish activity.	Returned to MainActivity with updated values.	**PASS**
T5	Activity Result API	Click "Edit Device", change fields, click Cancel.	Return RESULT_CANCELED; original data untouched.	UI displays original "Sensor-A" & "15%" values.	**PASS**
T6	Implicit Intent	Click "Open Documentation" button.	Open https://developer.android.com in browser.	System browser launches and loads URL.	**PASS**
T7	Lifecycle Observation	Navigate between screens and rotate device.	Logcat prints explicit callback traces.	Callback sequence logged accurately under tags.	**PASS**
T8	Input Validation	Submit empty name or battery value > 100.	Block submission; display inline error popup.	Validation triggers red inline error popups.	**PASS**

2. Intent Analysis, Extras, & Result Statuses

- **Explicit vs. Implicit Intents:** Explicit intents directly reference the target Java class (EditDeviceActivity.class), ensuring internal component execution. Implicit intents express a system-wide action (ACTION_VIEW) paired with a URI payload, letting Android's package manager resolve compatible system applications at runtime.
- **Extras Across Boundaries:** Intent extras act as serialized primitive Key-Value pairs wrapped inside an Android Bundle. They are passed across process/activity boundaries during IPC (Inter-Process Communication) and extracted using structured getters like getStringExtra() and getIntExtra().
- **RESULT_OK vs. RESULT_CANCELED:** RESULT_OK signals successful user action, triggering the ActivityResultLauncher callback in MainActivity to update state and refresh UI. RESULT_CANCELED signals cancellation or physical Back button presses, dropping changes and leaving caller state intact.

3. **Lifecycle Comparison:**

- *Navigation Transition:* Navigating to EditDeviceActivity causes MainActivity to execute onPause(), after which EditDeviceActivity completes onCreate() -> onStart() -> onResume(), and finally MainActivity completes onStop().
- *Configuration Change (Rotation):* Rotating the device causes full destruction and immediate recreation of MainActivity (onPause() -> onStop() -> onDestroy() -> onCreate() -> onStart() -> onResume()).

Diagnosed Defect: During screen rotation testing, updated values (Tablet-C, 45%) reverted to default values (Sensor-A, 15%). The defect was resolved by overriding onSaveInstanceState() to store values in the out-state Bundle and restoring them in onCreate() when savedInstanceState != null.

Student and environment	Details
Name and registration number	Esha Khalid, FA23-BCE-030
Android Studio / SDK / device	2026.1.4 / Medium Phone 37.1
Git commit ID / source reference	2a18e18

1. Critical Analysis

The implementation of Android component transitions using explicit and implicit intents highlights crucial structural trade-offs in mobile architecture. Explicit intents guarantee type-safety and internal control when transitioning between known screens. However, passing data via Extras requires strict coordination using shared constant keys (com.example.devicemanager.DEVICE_NAME) to avoid runtime payload mismatched types or NullPointerExceptionerrors. Using the modern registerForActivityResult API offers significant stability improvements over the deprecated onActivityResult pattern by decoupling result handling logic from monolithic activity overrides and preventing callback loss during activity recreation. Handling implicit intents requires defensive programming (try-catch with ActivityNotFoundException) to handle cases where no installed application matches the requested intent action. Furthermore, understanding activity lifecycle state transitions is vital; failure to explicitly preserve instance state during configuration changes leads to subtle state loss defects.

2. Summary of Skills learned

- Multi-Activity Navigation: Configured explicit intent transitions and data passing between Java activity components using structured Extras.
- Activity Result API: Registered and processed activity completion results (RESULT_OK / RESULT_CANCELED) using modern ActivityResultLauncher contracts.
- System Component Integration: Triggered implicit system actions (ACTION_VIEW) to open external web browser instances safely.
- Lifecycle Debugging: Monitored activity state transitions via Logcat tagging (MainActivityLC / EditDeviceActivityLC) across navigation and configuration change events.
- Input Validation & UX: Built robust input validation logic handling missing input, number formatting errors, and boundary limits.
- State Management: Implemented runtime state preservation across screen rotations using onSaveInstanceState()and Bundle restoration.

3. Conclusion

Lab 03 successfully demonstrated the primary mechanisms governing Android activity lifecycle management, inter-activity data transfer, and runtime UI state stability. By constructing the "DeviceManager" application, practical experience was gained in implementing explicit intents for structured data editing, implicit intents for external link navigation, and activity result callbacks for state synchronization. Identifying state reset defects during screen rotation underscored the critical importance of instance state preservation. All requirements T1 through T8 were implemented, validated, and verified, fulfilling the objectives of the lab exercise.